

The Speciating Island Model: An alternative parallel evolutionary algorithm

Steven Gustafson^{a,*}, Edmund K. Burke^b

^a*GE Global Research, Computing and Decision Sciences, Niskayuna, NY 12309, USA*

^b*University of Nottingham, School of Computer Science & IT, NG81BB, UK*

Received 3 May 2005; received in revised form 6 April 2006; accepted 17 April 2006

Abstract

This paper presents an investigation of a novel model for parallel evolutionary algorithms (EAs) based on the biological concept of species. In EA population search, new species represent solutions that could lead to good solutions but are disadvantaged due to their dissimilarity from the rest of the population. The Speciating Island Model (SIM) attempts to exploit new species when they arise by allocating them to new search processes executing on other islands (other processors). The long term goal of the SIM is to allow new species to diffuse throughout a large (conceptual) parallel computer network, where idle and unimproving processors initiate a new search process with them. In this paper, we focus on the successful identification and exploitation of new species and show that the SIM can achieve improved solution quality as compared to a canonical parallel EA.

© 2006 Elsevier Inc. All rights reserved.

Keywords: Parallel evolutionary algorithms; Genetic programming; Islands

1. Introduction

Evolutionary algorithms (EAs) are bioinspired methods that are naturally implemented in parallel models. EAs work by transforming a population of solutions (parents) into new candidate solutions (offspring) using a heuristic measure of solution quality (fitness) and transformation operators designed to allow offspring to inherit properties of the parent solutions (for example, two-parent crossover). EAs can be converted to work in parallel by distributing the independent algorithm tasks, such as fitness calculation, to separate processors. Parallel architectures for EAs are often the focus of the parallel computing community, for example for heterogeneous clusters [3] and large scale architectures [7]. However, EAs also conduct a distributed and parallel search in the solution space by concurrently searching with alternative solutions within the population of solutions. Given these two features of EAs, parallel computers can be used to achieve faster computation by distributed computing, or, parallel computers can be used to emphasise the biological inspiration of EAs: the concurrent search of solution alternatives

in the solution space. Parallel implementations of EAs typically realise the latter feature with structured population models, most notably the island model [1].

Structured population models are typically complex to implement and analyse, particularly with nontrivial solution spaces, and as such, they are often tuned to behave like the standard unstructured population EA. The original island model in EAs [6] was motivated by a biological theory suggesting that isolated subpopulations encouraged speciation events [8]. The metaphor of speciation is often touched upon in the EA literature, but the island model is not the ubiquitous implementation of that metaphor. Instead, the meaning and motivation of the island model is blurred with other forms of distributed computing, for example grid topologies, multipopulation models and demes.

This paper introduces and analyses the Speciating Island Model (SIM), briefly introduced in [10], that prescribes a niche for island models based on the metaphor of speciation. The SIM is based on the concept of species (outlier solutions in an EA) that are not able to mate effectively within the current population (produce fit solutions in an EA using transformation operators). The SIM aims to better utilise solutions encountered during the search process, which represent potential local/global optima, but that are typically ignored due to population convergence to other areas of the search space. While the intuition

* Corresponding author.

E-mail addresses: steven.gustafson@research.ge.com (S. Gustafson), ekb@cs.nott.ac.uk (E.K. Burke).

behind population-based search and island models is that the population is able to search different areas of the search space in parallel, results typically show, particularly for the genetic programming (GP) method, that this behaviour is limited and populations quickly converge to a similar solution structure and contents [10,21].

2. Background

The island model is an example of a distributed population model where subpopulations are isolated during selection, breeding and evaluation. Islands typically focus the evolutionary process within subpopulations before migrating individuals to other islands, or conceptual processors, which also carry out an evolutionary process. At predetermined times during the search process, islands send and receive migrants to other islands. There are many variations of distributed models, e.g. islands, demes, and niching methods, where each requires numerous parameters to be defined and tuned.

In [22], a distributed EA model used the concept of a species representing several *types* that are capable of mating and producing viable offspring. In [25], a parallel genetic algorithm (GA) was implemented on a hypercube structure. Random migrant selection and replacement was first introduced in [6]. The idea that islands should consist of distinctly different environments was seen in later work, often for coevolution. In [23], an architecture was used to adaptively co-evolve components in a speciation model. Subpopulations represented separate species, which were required to contribute to the overall fitness to survive. An injection island GA in [18] divided the search space into hierarchical levels, where populations trained on more general tasks are injected into populations with more specific tasks. The cohort GA [13] was designed to combat premature convergence, in the form of “hitchhiking” by allowing higher fit individuals to reproduce first, where offspring are assigned to cohorts based on their fitness. In [4], a multipopulation algorithm performed migration between subpopulations where individuals were redistributed to new subpopulations based on a speciation tree method that places genetically similar individuals together. A local GA was then run on each island for a number of generations to serve as an ‘intensification’ phase.

Much of the effort in distributed models in GP focus on increasing efficiency or adding computational resources by means of parallelisation. A typical example is found in [2], where each processor is responsible for the fitness evaluation and breeding of a subpopulation. Island models in GP are often considered as another form of multipopulation model, and in [9], various control parameters for multipopulation models are systematically studied. Multiple populations were also examined for GP in [24].

Just as multipopulation, distributed and island models often overlap with their goals and implementations, other forms of selection can also implement concepts of species and isolated subpopulations. For example, structure fitness sharing [14] was applied to GP to encourage the *parameter* search (functions and terminals) over similar *structures* (trees). Negative correlation [19] was used for learning ensembles of neural networks, where

populations were divided into species and a representative from each species was included in the ensemble. Negative correlation was also investigated for GP as a way to improve diversity and prevent premature convergence [20]. In [17], a “species conserving” GA defined species in a population using distance metric, where species “seeds” were individuals that were at least as fit or more fit than the rest of the species. These individuals were then “conserved”, or copied, into the next generation.

Parallel and distributed models for EAs can be generally split into two categories: those that implement a *panmictic* search process, and those that use a structured population. The reader is referred to [1] for a recent survey of parallelism and EAs. By panmictic, we refer to the biological concept of the entire population being capable of interbreeding. That is, spatial or other constraints do not prevent any solution with mating with another solution. In panmictic models, parallel and distributed implementations provide a means to speedup the EA by performing independent tasks in parallel over many processors. However, in structured populations, parallel and distributed implementations introduce constraints on the intermixing of solution that create a new type of EA. In this paper, we will consider an alternative to the panmictic model for parallel EAs.

Panmictic EAs are implemented in parallel in two main ways. The master–slave approach allows one EA run to be carried out in a shorter time than its sequential counter-part by assigning independent computations to processors working in parallel. Or, panmictic EAs simply assign each processor to run the same EA but with different initial conditions. In this case, collecting many independent runs of the EA buffers against stochastic effects and is often necessary for understanding the general behaviour of the EA. Both of the above approaches allow efficient execution of an EA on a parallel architecture. Both approaches do not change the dynamics of the EA run, but allow for many independent runs to be carried out in a time span similar to one sequential EA executing on one processor.

In [10], we defined solutions in a GP population with relatively good solution quality and high dissimilarity from the population as *outliers*. We measured the ability of the transformation operator to produce good solutions using outliers in combination with other population members. Using several common problem domains, the results showed that outliers were typically very poor at producing good solutions. However, infrequent events where outliers were far more successful than the rest of the population suggested that, under the right circumstances, the search process could be significantly improved by leveraging outliers more effectively. Thus, we were motivated to hypothesise that search may be improved by *speciating* outliers to new *subpopulations* or *islands* where they would *evolve* in *isolation*. That is, we would conduct a separate search with these fit and dissimilar solutions in parallel to the main GP search.

In summary, island models are used for a wide variety of purposes. From simple distributed computing to more advanced concepts of niches and hierarchical learning, island models are a convenient metaphor for EAs. In some EAs, for example GP, a niche may exist for island models to be cast into an explicit role to provide substantial and consistent improvements to the

search process. The goal of our research is to determine if such a niche exists and what are some possible ways of exploiting the niche. The benefit of explicitly casting the island model for such a task is that it can allow for a more unified study and advancement of the method. Otherwise, the island model remains a general tool for diversity management, premature convergence avoidance, and efficient parallelisation—goals that are shared by diversity methods, selection techniques, specialised recombination operators and many distributed and multipopulation models, e.g. see [5].

3. The speciating island model

The SIM has two primary goals. Firstly, it attempts to detect solutions that are sufficiently different from the current population to contribute effectively to the search process, but that have good enough solution quality to be considered in some alternative search environment. The island, or separate processing unit, provides a search process specifically designed for these solutions. Achieving this goal requires the understanding of population dynamics during search to identify outliers (new species) and to design a successful search method for them. Secondly, the SIM aims to provide a framework for a large network of processors. The idea is that an initial EA run is started on a processor, which diffuses newly arising species (or outliers) to neighbouring processors. Idle or unimproving processors then initiate a new search using these incoming species. This latter goal directly depends on the former one: the successful identification and exploitation of new species, which is the focus of this paper. The principles of the SIM are:

- (1) Solutions with good quality and an inability to contribute to search are *speciated*, where speciation allocates computational resources toward a new search process for these solutions on an idle processor (island).
- (2) Search processes diffuse new species throughout a network of islands, and idle and unimproving islands initiate new search processes with them.

Fig. 1 illustrates the proposed SIM, where search on an island halts due to nonimprovement and new species (outliers) are used to initiate new searches. In this way, it is hoped that islands can exploit new species and provide an efficient and simple way to perform dynamic resource allocation during the search process. In this study, we simplify this model to provide the circumstances for meaningful comparisons with alternative approaches.

Later, in the experimental study, we use 8 instances of a tunably difficulty problem for the GP, with 240 runs over those instances. In any one of the populations occurring during a GP run, we can count the number solutions that are dissimilar from the population (for example, using an edit distance-based measure for similarity, solutions with a pair-wise distance greater than the populations mean pair-wise distance) and with relatively good solution quality (for example, that have a better solution quality than more than half the population). These solutions will represent *outlier* solutions. Using the populations from these experiments that occur at generation 15, the

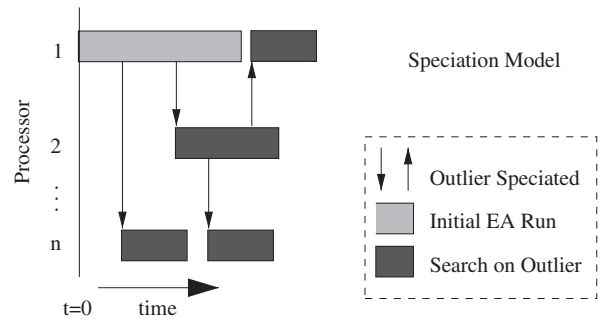


Fig. 1. Proposed island model evolutionary process with dynamic resource allocation.

average distribution of pair-wise distances from those populations is shown in the right-hand graph of Fig. 2. Fig. 2 (left-hand graph) shows a box and whiskers plot for the average number of *Outliers* in generation 15. We also report the average number of offspring these solutions produced (*Produced*), and then the average number of times those offspring were selected to produce an offspring in the following generation (*Survived*). We can see that the offspring produced by outliers tend to produce fewer offspring themselves, but with a surprisingly wide-range. These results can be expected when the population converges away from outlier solutions. The wide-range of *Survived* in Fig. 2, albeit heavily skewed toward 0, suggests that outliers are capable of providing significantly good solutions under some circumstances.

In [11], we began to validate the SIM. We found that using a nuanced definition of outlier solutions and a simple hill-climbing search on those solutions improved the final solution quality of the GP run. Several questions were left unanswered in that study that are addressed now:

- Using the criterion in [11] for selecting outliers often resulted in no solutions being selected, due to populations having different distributions of dissimilarity and fitness. Here, we present a more well-grounded approach that is predictable and matches the changing dynamics of the population. This approach for identifying outliers (new species) will be more suitable for the second goal of the SIM.
- In [11], we were only concerned with understanding if the SIM produced *any* improvements for one run. Here, we compare our approach to a multirun GP with a panmictic population structure, the most commonly type of GP. This comparison will help us understand if the SIM can compare to the panmictic population alternative of collecting many independent runs, in terms of solution quality and search complexity.
- We improve upon the study in [11] by comparing speciation using outliers against speciation using randomly selected solutions, solutions selected according to their fitness and solutions selecting with knowledge of the problem.
- We also compare the SIM to a multirun EA, using the same number of solution evaluations but with smaller and larger population sizes.

The proposed SIM is intended to be a viable alternative to simple panmictic parallel EAs, hopefully leading to a new

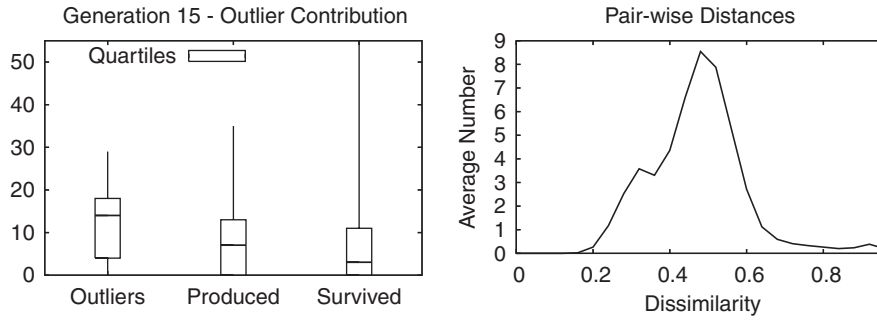


Fig. 2. In the left-hand graph, the number of *Outliers* at generation 15, number of offspring *Produced* by those outliers, and the number of those offspring that *Survived* to produce offspring in the next generation. The graph on the right shows the average distribution of pair-wise distances at generation 15.

niche for island models. In the next section, we carryout an experimental investigation, where the implementation of our SIM is meant to be as general as possible while conveying its main idea.

4. Experimental verification

4.1. The Tree-String problem

The Tree-String problem is an artificial domain constructed to capture two important features of GP search: solution structure and content [10,12]. GP is an EA that uniquely searches for the structure and content of solutions at the same time. The goal of the Tree-String problem is to derive specific structure and content elements simultaneously. Instances are defined using a target solution consisting of a tree shape and content. Candidate solutions are measured for their similarity to the target solution with respect to both tree shape and content. The Tree-String problem is defined as a tuple $\Pi = (\Psi, \Xi, t, \alpha, \gamma, \delta)$, where an instance is represented by a target solution t , composed of content elements from the set Ψ and has a tree shape defined by elements from the set Ξ . In this study, and in [11,12], we use the following:

- $\Xi = \{n, l\}$, representing *nodes* and *leaves* in binary syntax trees,
- $\Psi = \{A, \dots, C\}$, representing both node and leaf labels,
- $\alpha(t) \mapsto \Xi^*$, a breadth-first tree traversal over solution structure (tree shape) creating the structure string,
- $\gamma(t) \mapsto \Psi^*$, a depth-first, in-order tree traversal over solution content creating the content string,
- $\delta(\alpha(t_i), \alpha(t_c)) \mapsto i \in \mathbb{N}$, and $\delta(\gamma(t_i), \gamma(t_c)) \mapsto j \in \mathbb{N}$, where i and j represent the heuristic solution quality of t_c compared to instance t_i , and δ is the longest common substring function, and
- the fitness of a candidate solution is the linear combination of objective functions, i.e. $fitness = i + j$.

In the Tree-String problem, as defined above, the portion of the solution that contributes to the structure objective is likely to be different from the part that contributes toward the content objective. This property is due to the use of the breadth-first traversal for the structure objective and the depth-first traversal

for the content objective. The fact that the two objectives are interdependent is likely to make it difficult for transformation operators to affect either content or structure objectives alone.

These two implicit solution features of GP search, structure and content, are difficult to address directly in most benchmark and real-world problem domains. Thus, the Tree-String problem provides a domain in which structure and content can be explicitly controlled and measured. Augmenting GP search with methods intended to specifically address the structure and content objectives is akin to adding highly specific domain knowledge. While we use a standard GP system and a linear combination of structure and content objectives for search, we compare GP performance using a metric that is aware of both objectives.

We create instances in the Tree-String problem with an increasing size (number of nodes) at the same depth and increasing size with increasing depth. To create an instance, we generate a tree shape using an iterative tree growth method and then assign random content to the tree. The tree growing method iteratively adds two child nodes to a probabilistically chosen leaf node, starting with the root. To produce 500 random trees with depths between 5 and 15, and with size between 15 and 272 nodes, we: (1) randomly pick a tree size from the latter range, and (2) iteratively grow a tree shape to that size with a limit of depth 15. The 500 random trees shapes created are shown in Fig. 3 according to their depth and size. We select tree shapes from depths 7, 9, 11, and 13, labelled as $\tau^{1,4,7,8}$ in Fig. 3. We also select tree shapes from depth 9 with increasing sizes, labelled as $\tau^{2,3,5,6}$ in Fig. 3. These trees are shown in Fig. 3 using a circular lattice visualisation. The root node lies at the very center, and each two child nodes lie at the intersection of subsequent lines. The inner-ring marks the maximum depth of that tree. The second step to define our instances is creating the content that each tree shape will have. In this study, we only consider the increase in instance difficulty according to instance size. Thus, we create a random string for each tree shape using three unique symbols: *A*, *B*, and *C*. The content generation algorithm, for a tree size of n nodes, simply picks n times a random symbol from $A \dots C$. Each random string is the same size as the tree shape under consideration—producing 8 instances. The GP system will use the same

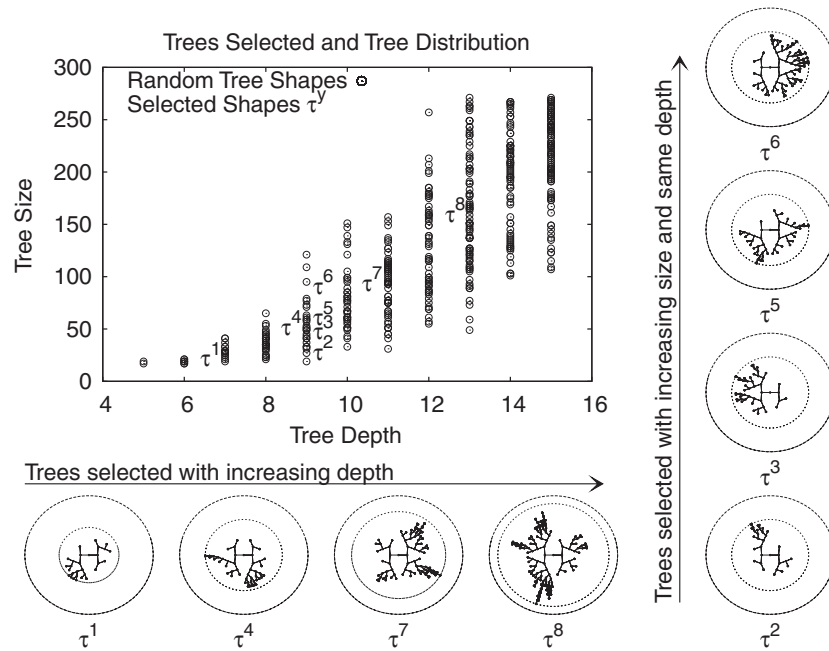


Fig. 3. The 500 tree shapes produced are plotted according to their depth and size (number of nodes).

content set ($A \dots C$) as used to create the current instance under consideration.

4.2. The GP system

The GP algorithm uses a generational replacement of its solutions, with the two-parent subtree crossover to transform parent solutions into new offspring solutions. Two parent crossover selects a subtree (where nonleaf nodes are selected 90% of the time) from each parent and swaps them. All children are valid provided they are within a depth limit of 20. To select a parent for crossover, tournament selection randomly selects 8 solutions and keeps the best. The initial population is created by producing random trees using the Full and Grow methods equally between depths 2 and 4. A population size of 105 and a stopping criterion of 50 generations is used. Fitness is based on the minimisation of the objective function, described in Section 4.1.

4.3. Population size and selection pressure

Using one instance of the problem (τ^3), we executed 10 random trials of the GP system with population sizes between 50 and 450, with increments of 100, and with tournament selection size between 2 and 10, with increments of 2. Fig. 4 reports the average objective function values for each setting. It is clear from Fig. 4 that GP responds favourably to larger populations and larger tournaments. To choose a favourable selection pressure, we use a tournament size of 8. However, to provide grounds for a useful comparison later, a population size of 105 is used to determine the performance of the speciation model against various forms of selecting new “species”.

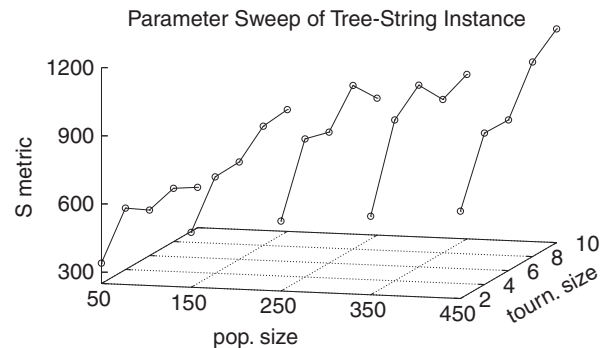


Fig. 4. The performance of the GP system over various population and tournament sizes.

4.4. Comparison measures

We treat the multiobjective nature of the Tree-String problem linearly in the fitness function. This represents most closely the purpose of the Tree-String problem: to allow for explicit control and study of the two implicit search objectives (structure and content) in standard GP search that are typically very difficult to control and measure directly. While the objective values are combined linearly in the fitness function, we can still measure the performance in terms of both objectives and compare experiments accordingly. It is particularly advantageous in this instance, where we are concerned with how effectively the search space is sampled, to consider both objectives independently. Thus, as discussed in [15,16], we use the S -metric for comparing two Pareto fronts defined by the objective functions (two in this paper). The Pareto front of a set of solutions consists of those solutions for which no other solution exists in the

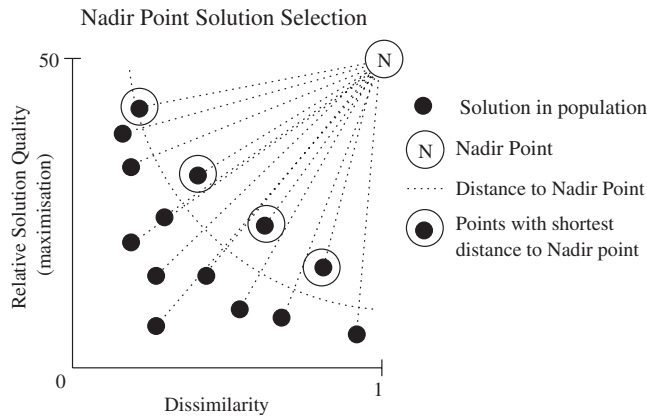


Fig. 5. The Nadir method for selecting outliers. The points with the shortest distance to the Nadir point are chosen as outliers. If more than the required number of points have the same distance, they are selected at random.

set that is better in at least one objective and no worse than the other objectives. The *S*-metric computes the area, in two dimensions (or volume in greater dimensions), defined by the Pareto front and some predefined reference point, Z_{ref} . Fig. 6, Right-hand graph, demonstrates this area. In this paper, the value of z_{ref} is set to be the size of the instances, as this is the maximum values of either objective. For the remainder of the paper, the term *Pareto front* will be used exclusively with respect to the set of nondominated solutions in the *objective* space.

4.5. Outlier definition

We define outliers as dissimilar, or genetically different, from the rest of the population according to an edit distance. The edit distance is defined as follows: two trees are overlapped at the root, and the number of node transformations, insertions and deletions that are required to make the two trees structurally and syntactically equal are counted. The distance is normalised by tree size. An individual's pair-wise distance is the average of all the normalised distances to the rest of the population. We then rank solutions according to the number of solutions from the population that they are better-than in solution quality. The average pair-wise dissimilarity and relative quality of a solution provide criterion for which outliers can be chosen. To accomplish this, we use the concept of a *Nadir* point, as shown in Fig. 5. As we know the maximum dissimilarity (1.0) and the maximum relative solution quality (104, for a population size of 105 solutions), we can measure the distance from the Nadir point ([1, 105]). A shorter distance implies greater values in both objectives. This encompasses the concept of selecting from the set of nondominated solutions, while allowing for a fixed number of solutions to be denoted as outliers based on their distance to the Nadir point, regardless if a particular set of nondominated solutions contains as many. We also scale the dissimilarity value to also be in the range of [0, 105] so as to give similar weight to both objectives. At generation 5, 15 and generation 30, we select 5 outliers according to the Nadir point method.

4.6. Speciation of solutions

To *speciate* a solution to a new island requires the initialisation of a new EA with respect to that solution. Of course, specific methods can be used, but for the sake of generality, we use the following method. An identical GP system as to the one that produced the solution speciated, is initialised. However, the initial population is produced by performing random mutations to the speciated solution, which may or may not be present in the population according to the behaviour of the mutation operator. Mutations consist of randomly created subtrees of depth between 1 and 3, replacing subtrees in the speciated solution located at a leaf position (with probability 0.5), at a nonleaf and nonroot position (with probability 0.4) or at the root position (with probability 0.1). If mutation replaces a subtree with an identical subtree, or fails to produce a new solution under depth 20 in 100 trials, the speciated solution is copied into the new population. All other GP parameters remain the same, and each speciated solution is allocated one GP run consisting of 51 generations.

4.7. Definition of experiments

Along with speciating outliers, we also speciate (at generations 5, 15, and 30) 5 random solutions, the 5 fittest solutions, and 5 solutions with the longest unique vector to a Nadir point (defined in this case by the maximum values of the objective functions) from a solution objective values (i and j). Once all vector lengths to the Nadir point have been calculated, the five longest unique vectors (and their solutions) are selected, where a solution is selected randomly when more than one solution has one of the five longest vector lengths. This latter experiment is akin to selecting nondominated solutions, where a longer vector to a Nadir point denotes a solution with maximal matching to the target in one or both objectives. We label these experiments as A_O (outlier speciation), A_R (random speciation), A_F (fittest speciation) and A_P (Pareto-based speciation). Each of these experiments are carried out for 30 random trials using the same original 30 random runs over each of the 8 Tree-String instances.

We also carryout several multirun GP experiments, consisting of roughly the same number of solution evaluations as the speciation experiments. These experiments use the exact same GP system, but vary the population size and the number of runs. They are defined as B_{52} (population size of 52 solutions and 32 random runs), B_{105} (105 solutions and 16 runs), B_{210} (210 solutions and 8 runs) and B_{1050} (1050 solutions and 2 runs). Each of these experiments is also carried out for 30 random trials on each of the 8 Tree-String instances. For example, in the case of B_{210} , the average performance of the 30 trials, consisting of 8 runs each, is compared to the average performance of 30 trials of a speciation experiment, consisting of 1 run plus speciation events.

5. Results

The two key objectives of the experimental study is to verify that our general definition of outliers is useful for speciation

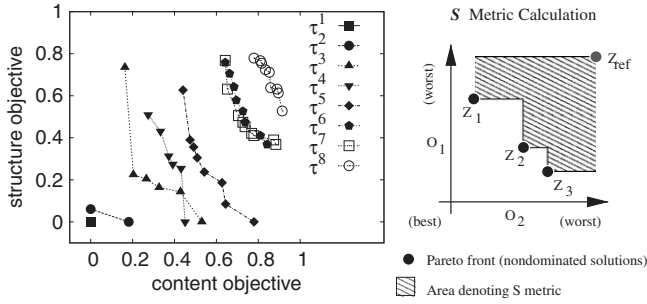


Fig. 6. On the left, the Pareto front of nondominated solutions from a selection of experiments for each of the 8 instances. The objective values are normalised by instance (tree) size. On the right, an example of the S -metric for comparing Pareto fronts using two objectives o_1 and o_2 , three nondominated solutions (z_1, z_2, z_3), and one reference point z_{ref} .

and that speciation of outliers can result in better performance over a multirun approach. Thus, in Section 5.1, we examine the differences between speciating various solutions (experiments A_O , A_R , A_F , A_P), and their ability to improve the original runs. In Section 5.2, we examine the ability of 1 run plus speciation to improve over a variety of parallel, multirun EAs (experiments B_{52} , B_{105} , B_{210} , B_{1050}). Fig. 6 demonstrates the ability of the GP system to improve fitness on the Tree-String instances. As instance size increases, the amount of improvement GP makes decreases. Remember that while Fig. 6 reports the Pareto front of nondominated solutions, the actual GP algorithm only deals with the objective values in the linear fitness function.

5.1. Speciation and outlier definition

We first examine the ability of speciation to improve the original run, and the amount of improvement achieved. That is, the original runs' S -metric is compared to the S -metric achieved by speciating 5 solutions at generation 5, 15 and 30. Table 1 reports the number of runs improved by speciation for each speciation experiment, as well as the average percentage improvement when speciation did improve the original run's S -metric. All experiments improved relatively the same number of runs. The A_O experiment nearly always has a higher average percent improvement, sometimes a significant one. The A_O and A_P are more similar in their performance.

We compare the average S -metric obtained by the 30 runs of all speciation experiments *without* considering the original run from which speciation occurred. Note that the same original 30 runs were used to provide solutions to be speciated for all experiments. The differences arise due to the selection of solutions to speciate as well as the stochastic behaviour of the algorithm. Fig. 7 shows a box-and-whiskers plot of the median, first and third quartiles, and the minimum and maximum S -metric values, where a T-test comparison of this data is reported in Table 2. The smallest (easiest) instance τ^1 shows all experiments achieve close to optimal results. For all other instances, A_O is generally better than the A_R and A_F experiments, and again, very competitive with the A_P experiment. Speciating randomly selected solutions is similar to speciating Pareto-based

Table 1

The number runs (#) improved by speciation (out of 30) and average percentage (%) improvement of speciation over original runs

	A_O		A_R		P	A_F		P	A_P		P
	#	%	#	%		#	%		#	%	
τ^1	15	22.3	15	23.0	=0.08	15	23.7	=0.14	15	23.7	=0.15
τ^2	29	59.7	30	49.7	=1.01	29	48.7	=1.28	28	63.2	=0.14
τ^3	29	90.8	30	75.3	=0.97	28	70.2	+1.72	29	76.6	=1.05
τ^4	28	63.6	29	57.0	=0.48	26	55.7	=1.16	28	67.5	=0.37
τ^5	29	67.0	29	67.0	=0.00	29	55.1	=1.17	30	82.2	=1.47
τ^6	30	80.8	30	61.1	+2.02	25	48.9	+4.07	30	77.2	=0.35
τ^7	29	111.0	30	98.6	=0.52	30	92.5	=0.95	28	111.8	=0.17
τ^8	28	97.7	29	81.0	=0.99	29	74.3	=1.64	29	86.0	=0.56

A t -test comparison between the percent improvement for A_O is compared against that achieved by A_R , A_F , A_P . A "=" denotes no-significant difference, "+" denotes A_O was significantly better, and "-" denotes A_O was significantly worse. Significance was tested at the 0.1 level and P -values are reported.

solutions and outliers. Intuitively, the highest-fit solutions are more likely to get a speciation search stuck in a local optimum than random selected solutions or ones picked with full knowledge of the structure of the fitness function (Pareto-based).

The contributions of speciating solutions at the three different generations (5,15,30) is examined in Table 3. Interestingly, A_O is never statistically worse than the other experiments at any generation and for any instance. In several places, e.g. instances τ^6 and τ^7 , A_O is statistically better than the A_F and A_R experiments. Speciating solutions at generation 5, 15 and 30 results in the highest average S -metric in 4/8 instances when those solutions are outliers, but not necessarily the same instances for the different generations. *No other method, including A_P , achieves an equal or better result in this last respect.*

In Table 4, we gather all the solutions found in all 30 runs of each experiment and calculate the Pareto front based on the two objective functions and the corresponding S -metric. We do this to understand the type of solutions found. As expected, the outlier experiment achieves a greater S -metric. Intuitively, this is due to the speciation of solutions that lead to different points on the Pareto front. The outlier solutions encourage this behaviour as they speciate good solutions that are also dissimilar, whereas the other speciation experiments use highly fit only or random solutions.

5.2. Speciation performance

We now compare the average S -metric obtained by the 30 runs of the outlier speciation to the different multirun approaches. We expect these multirun approaches to be very competitive, simply due to the stochastic nature of the algorithm coupled with the high number of random runs each performs, and also due to the positive reaction to an increase in population size seen in Fig. 4.

Fig. 8 shows a box-and-whiskers plot, and Table 5 reports a T-test comparison, for the A_O and $B_{52...1050}$ experiments. It is clear that A_O achieves higher S values than B_{52} . For several instances, $\tau^{1,3,7,8}$, A_O is similar to B_{105} . Note that B_{105} consists of 30 experiments, where each experiment reports the

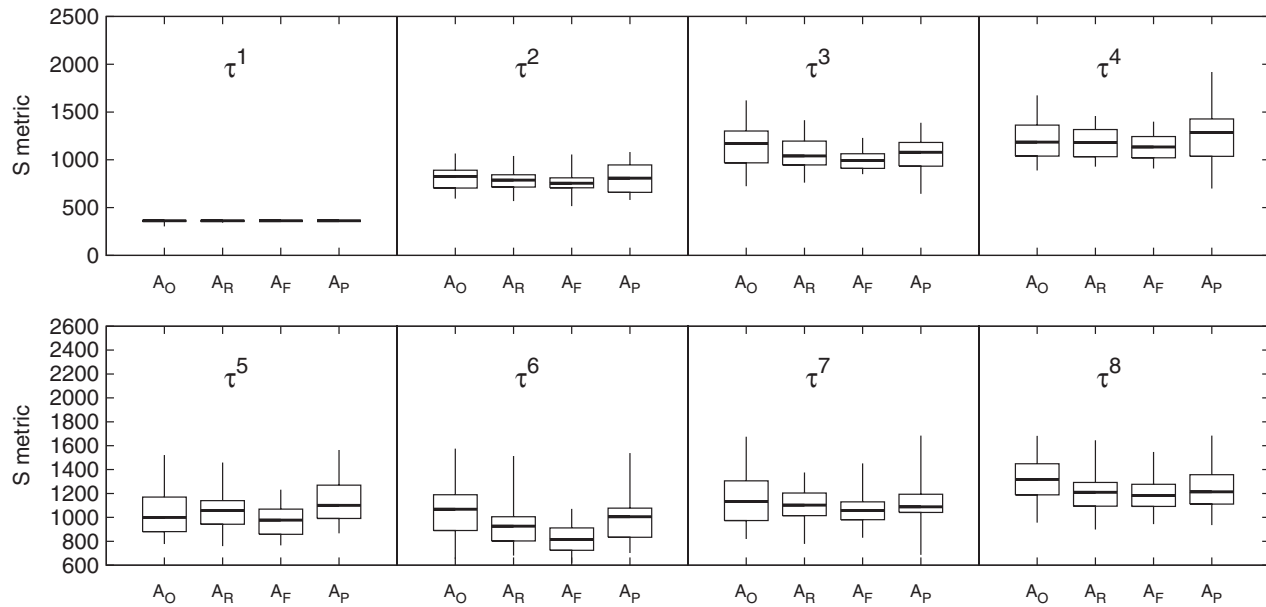


Fig. 7. The distribution of S -metric values achieved during the speciation experiments. Table 2 reports a t -test for these experiments.

Table 2

The t -test comparison for experiments reported in Fig. 7 between the S -metric achieved by the speciation experiments, where A_O is compared against A_R , A_F , A_P

	τ^1	τ^2	τ^3	τ^4	τ^5	τ^6	τ^7	τ^8
t -test between A_O and other experiments reported in Fig. 7								
A_R	=0.56	=1.29	=1.28	=0.55	=0.37	+2.36	=1.27	+1.90
A_F	=0.98	+1.97	+2.79	+2.25	+1.80	+5.29	+1.77	+2.77
A_P	=1.00	=0.07	=1.36	=0.36	=1.67	=0.36	=0.52	=1.36

A “=” denotes no-significant difference, “+” denotes A_O was significantly better, and “-” denotes A_O was significantly worse. The P -values are also reported, where significance was tested at the 0.1 level.

S -metric for the combination of all solutions from 16 random runs. Similarly, A_O also consists of the S -metric from 16 runs (1 original run plus 15 speciation runs). However, the A_O runs are not all independently random as the 15 speciation runs all *depend* on the 1 original run. This is a promising result with respect to the goal of the speciation model: one run can speciate outliers and achieve a similar overall result as that achieved by carrying out many randomly independent runs. With respect to

Table 4

The S -metric for all runs in a given experiment

	τ^1	τ^2	τ^3	τ^4	τ^5	τ^6	τ^7	τ^8
A_O	361	1081	1750	1758	1839	1692	1841	2022
A_F	361	1053	1513	1620	1733	1533	1725	1863
A_R	361	1056	1375	1590	1596	1251	1574	1750
A_P	361	1083	1683	1970	1734	1752	1732	2022

For example, all solutions from all 30 runs of experiment A_O are combined, and the resulting Pareto front and S -metric are calculated.

the larger population experiments $B_{210,1050}$, Fig. 8 shows that speciating outliers does overlap in performance with both these experiments to some degree. As we expect GP to perform better with larger populations (see Fig. 4) on the Tree-String problem, these results certainly suggest that speciating outliers, possibly with a better initial population technique for speciation or other parameters, could outperform these experiments.

Lastly, Table 6 reports the S -metric for the combination of all solutions from all runs in each experiment. The A_O experiment

Table 3

Average S -metric achieved by the speciation experiments at each speciation generation (5, 15, 30)

	τ^1			τ^2			τ^3			τ^4			τ^5			τ^6			τ^7			τ^8		
	g5	g15	g30	g5	g15	g30	g5	g15	g30	g5	g15	g30	g5	g15	g30	g5	g15	g30	g5	g15	g30	g5	g15	g30
A_O	351	357	356	737	720	721	931	986	919	1022	1022	1062	888	858	899	883	900	884	919	1003	984	1093	1077	1048
A_R	342	348	349	660	661	670	873	925	863	1014	1007	990	907	855	855	727	795	780	904	896	904	974	1063	986
A_F	353	354	354	654	645	616	846	799	812	907	914	891	838	802	805	684	692	664	865	858	854	992	904	973
A_P	358	356	360	709	720	711	917	901	908	1102	1081	1109	970	995	932	850	901	875	934	924	920	1017	1035	1024

When A_O is significantly better than A_R , A_F , or A_P , using a t -test at the 0.1 level, the value of the latter experiment is reported in *bold*. Otherwise, A_O is statistically equal, at the 0.1 level, to the other experiments (i.e. never worse).

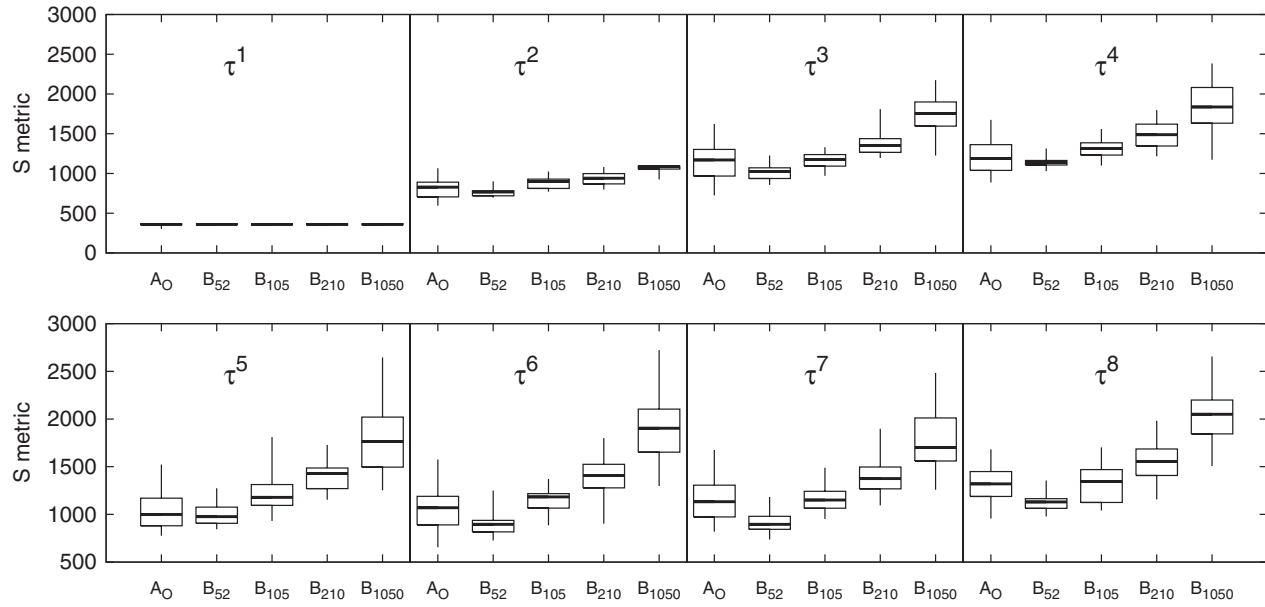


Fig. 8. The distribution of S -metric values achieved during the 30 experiments of speciation with outliers (A_O) and the multirun approaches ($B_{52} \dots B_{1050}$). Table 5 reports a t -test for these experiments.

Table 5

The t -test comparison for experiments reported in Fig. 8 between the S -metric achieved by the speciation experiments, where A_O is compared against A_R , A_F , A_P

	τ^1	τ^2	τ^3	τ^4	τ^5	τ^6	τ^7	τ^8
<i>t</i> -test between A_O and other experiments reported in Fig. 8								
B_{52}	=1.02	+2.26	+2.75	+2.23	=1.46	+3.63	+4.85	+4.90
B_{105}	=1.02	-2.21	=0.62	-2.22	-3.05	-2.44	=0.21	=0.51
B_{210}	=1.02	-4.25	-4.70	-5.75	-7.32	-6.64	-4.64	-5.34
B_{1050}	=1.02	-9.48	-9.96	-10.48	-10.70	-12.73	-8.79	-12.07

A “=” denotes no-significant difference, “+” denotes A_O was significantly better, and “-” denotes A_O was significantly worse. The P -values are also reported, where significance was tested at the 0.1 level.

Table 6

The S -metric for all runs in a given experiment

	τ^1	τ^2	τ^3	τ^4	τ^5	τ^6	τ^7	τ^8
A_O	361	1077	1799	1708	1672	1719	1774	2142
B_{52}	361	963	1349	1374	1603	1430	1453	1673
B_{105}	361	1063	1533	1635	2008	1748	1839	1962
B_{210}	361	1085	2006	1861	2078	2334	2081	2236
B_{1050}	361	1089	2295	2385	2823	3208	2929	2990

For example, all solutions from all 30 runs of experiment A_O are combined, and the resulting Pareto front and S -metric are calculated.

is now very competitive with the B_{210} experiment on most instances. For example, for τ^8 , the maximum S -metric is (163×163) 26569. The difference between A_O and B_{210} is only 94 (i.e. $2236 - 2142$), which is roughly the area of a 9×9 rectangle in the two dimensional space (163×163) where the S -metric is calculated.

5.3. Comparison with the linear objective function

We have measured the success of the algorithms in this paper according to the S -metric. We have done so to capture the implicit search for solution structure and content. The Tree-String problem allows us to artificially select a particular target structure and content and measure a solution’s fit to those targets. The domain knowledge of the target’s structure and content was kept hidden from the search algorithm, where the objective functions were combined linearly. The only place this domain knowledge was used was in the A_P experiment, i.e. Pareto-based speciation experiment. In the outlier selection process (where outliers are speciated), outliers were selecting according to the linear objective value.

The S -metric describes the hypervolume defined by a set of nondominated solutions and a predefined point (the maximum objective values in this paper). While a GP run’s S -metric is obviously correlated to its linear objective function value, when ranking the performance of several runs using the S -metric and the linear objective function, differences may arise. Therefore, to present a complete picture of the methods in this paper, we report in Table 7 the average best fitness (linear objective value) of the A_O , A_R , A_F , A_P experiments and the B_{52} , B_{105} , B_{210} , B_{1050} experiments. Note that, with respect to the linear objective function, *smaller fitness values are better*. Similar to the results using the S -metric, the A_O experiment was no worse than the other speciation experiments and sometimes better. Also as the instance size increases, the larger populations in the B_{52} to B_{1050} experiments are more beneficial. Interestingly, in the very large population experiment (B_{1050}), there is evidence (the decreasing P -values) to suggest that as the instance size increases, speciating outliers is beneficial even in the linear objective function space. This may be due to the

Table 7

The average best fitness of all experiments is reported

	A_O	A_R	A_F	A_P	B_{52}	B_{105}	B_{210}	B_{1050}
τ^1	0.87	0.92=0.16	0.81=0.20	0.83=0.11	0.00–3.79	0.00–3.79	0.00–3.79	0.00–3.79
τ^2	17.25	18.76=1.54	19.42+2.33	16.84=0.40	17.51=0.27	13.29–3.92	9.72–7.91	1.86–16.09
τ^3	42.40	44.28=1.39	44.97+2.06	44.06=1.15	44.21=1.47	39.57–2.34	33.78–6.33	21.11–14.10
τ^4	41.30	42.45=0.85	44.26+2.48	40.70=0.41	40.95=0.33	35.96–4.62	30.22–9.64	17.59–16.10
τ^5	63.98	64.42=0.32	64.92=0.68	62.55=1.06	62.96=0.80	58.05–4.45	52.23–8.46	39.37–14.27
τ^6	135.97	139.02+2.09	142.25+4.92	136.12=0.10	137.17=0.97	130.13–4.52	123.96–8.15	109.74–15.37
τ^7	135.05	135.94=0.71	135.95=0.70	136.81=1.33	137.86+2.29	131.55–2.70	126.12–6.99	114.99–13.43
τ^8	266.41	267.27=0.61	268.34=1.40	267.46=0.75	268.09=1.33	260.15–4.08	256.38–7.92	243.32–13.85

Smaller fitness values are better. The outlier speciation experiment (A_O) is compared to the other experiments using a *t*-test. Significance was tested at the 0.1 level and *P*-values are reported next the compared-to experiment, where a “=” denotes no-significant difference, “+” denotes A_O was significantly better, and “–” denotes A_O was significantly worse.

larger (and more difficult) instances having more local optima, which speciating outliers may avoid better than increased population sizes.

6. Discussion and conclusions

Previous research has employed the concept of species and island models for a variety of purposes. Most commonly, these methods are used to encourage populations to concurrently occupy a variety of different areas of an EA search space. For example, in fitness sharing, solutions with a similar fitness are penalised as it is thought they also occupy a similar area of the search space. Other methods like structured population parallel models are less rigorous in explicitly denoting whether two subpopulations do or do not occupy a similar area of the search space. In this paper, we proposed a SIM that was directly grounded in recent GP research.

First, GP populations tend to converge according to edit distance and lose most dissimilarity between population members [5]. Secondly, using a two-parent transformation operator, dissimilar solutions with good solution quality were unable to be consistently used to produce new solutions with relatively good quality [10]. These two trends suggested that a general way to improve EA search, particularly in the GP representation, is to exploit these dissimilar solutions with good quality (new species, or outliers) in a search process where they may be able to produce good offspring.

We intend for the SIM to be an alternative to parallel EAs where species can be searched in parallel to the original run. To make maximum use of a set of processors, and the potential for exploiting new species, we aim for the proposed model to be a framework for managing long, continuous EA searches. By diffusing new species to processors as they arrive, processors will always have a repository of potential seeds for which to initiate a new search process. These seeds (or new species from other searches) represent somewhat already optimised solutions, thus the search is not starting from a random point in the search space. Obviously, this latter goal of the SIM is not trivial, but it does primarily depend on the successful identification and exploitation of outlier solutions.

We have carried out an empirical study to validate the concept of speciating solutions. This study provided positive results

when comparing speciation using outliers (defined by dissimilarity and fitness) against speciation using randomly selected solutions, solutions selected according to fitness and solutions selected according to domain knowledge. The proposed speciation model was also compared against a parallel EA. Results in this latter study showed the ability to achieve good results using one original EA run with speciation in comparison to performing many randomly independent runs.

In conclusion, this paper presented an alternative to the panmictic structured population model for parallel EAs. The alternative is a Speciating Island Model (SIM) that applies a separate search method to solutions that represent new species due to their dissimilarity and fitness as compared to the rest of the population. In previous work, these solutions were shown not to be able to produce good offspring within the context of the current GP population. The SIM is intended to fill a niche in EA search by casting the island model in the role of detecting dissimilar and fit solutions (outliers) and exploiting them with a concurrent search method. We tested the SIM against standard EA runs, several multirun parallel EAs, and speciation using other forms of selecting speciated solutions. The proposed model consistently improved results and showed promise over increasing instance size. The experiments used to make these conclusions came from a canonical system, with little to no algorithmic tuning to achieve results, applied to a range of difficult instances on a problem designed to capture essential features from many commonly used and other artificial domains. Implementing our long-term goal of long, continuous runs using speciation events to exploit outlier solutions and make efficient use of large parallel machines will be a challenge to the EA and parallel computing communities.

Acknowledgment

This work was supported by the Engineering and Physical Sciences Research Council (EPSRC), Grant GR/S70197/01.

References

- [1] E. Alba, M. Tomassini, Parallelism and evolutionary algorithms, *IEEE Trans. Evolutionary Comput.* 5 (2002) 443–462.

- [2] D. Andre, J.R. Koza, Parallel genetic programming: a scalable implementation using the transputer network architecture, in: P.J. Angeline, K.E. Kinnear, Jr. (Eds.), *Advances in Genetic Programming*, vol. 2, The MIT Press, Cambridge, MA, USA, 1996 (Chapter 16).
- [3] V.E. Bazterra, M. Cuma, M.B. Ferraro, J.C. Facelli, A general framework to understand parallel performance in heterogeneous clusters: analysis of a new adaptive parallel genetic algorithm, *J. Parallel Distributed Comput.* 65 (1) (2005) 48–57.
- [4] M. Bessaou, A. Pétrowski, P. Siarry, Island model cooperating with speciation for multimodal optimization, in: M. Schoenauer, et al. (Eds.), *Parallel Problem Solving from Nature*, Paris, France, 2000, Springer, pp. 437–446.
- [5] E.K. Burke, S. Gustafson, G. Kendall, Diversity in genetic programming: an analysis of measures and correlation with fitness, *IEEE Trans. Evolutionary Comput.* 8 (1) (2004) 47–62.
- [6] J.P. Cohoon, S.U. Hegde, W.N. Martin, D. Richards, Punctuated equilibria: a parallel genetic algorithm, in: J.J. Grefenstette (Ed.), *Proceedings of the Second International Conference on Genetic Algorithms*, Hillsdale, NJ, USA, 1987, Lawrence Erlbaum Associates, Hillsdale, NJ, 1987, pp. 148–154.
- [7] S.E. Eklund, A massively parallel architecture for distributed genetic algorithms, *Parallel Comput.* 30 (2004) 647–676.
- [8] N. Eldredge, S.J. Gould, *Punctuated Equilibria: An Alternative to Phyletic Gradualism*, Freeman, Cooper and Co., San Fransico, CA, 1972, pp. 82–115 (Chapter 5).
- [9] F. Fernandez, M. Tomassini, L. Vanneschi, An empirical study of multipopulation genetic programming, *Genetic Program. Evolvable Mach.* 4 (1) (2003) 21–51.
- [10] S. Gustafson, An analysis of diversity in genetic programming, Ph.D. Thesis, School of Computer Science and Information Technology, University of Nottingham, Nottingham, England, February 2004.
- [11] S. Gustafson, E.K. Burke, A niche for parallel island models: outliers and local search, in: E. Cantú-Paz, F.F. de Vega (Eds.), *First International Workshop on Parallel Bioinspired Algorithms*, Oslo, Norway, June 2005, IEEE Computer Society, Silver Spring, MD, 2005.
- [12] S. Gustafson, E.K. Burke, N. Krasnogor, The tree-string problem: an artificial domain for structure and content search, in: M. Keijzer et al. (Eds.), *Genetic Programming. Proceedings of the Sixth European Conference*, Lecture Notes in Computer Science, vol. 3447, Lausanne, 2005, Springer, Berlin, 2005, pp. 215–226.
- [13] J.H. Holland, Building blocks, cohort genetic algorithms, and hyperplane-defined functions, *Evolutionary Comput.* 8 (4) (2000) 373–391.
- [14] J. Hu et al., Structure fitness sharing (SFS) for evolutionary design by genetic programming, in: W.B. Langdon et al. (Eds.), *Proceedings of the Genetic and Evolutionary Computation Conference*, Los Altos, CA, 2002, New York, 9–13 July 2002. Morgan Kaufmann Publishers, pp. 780–787.
- [15] J. Knowles, Local-search and hybrid evolutionary algorithms for pareto optimization, Ph.D. Thesis, Department of Computer Science, University of Reading, UK, 2002.
- [16] J. Knowles, D. Corne, On metrics for comparing non-dominated sets, in: *Proceedings of the Congress on Evolutionary Computation*, IEEE Press, New York, 2002, pp. 711–716.
- [17] J.-P. Li, M.E. Balazs, G.T. Parks, P.J. Clarkson, A species conserving genetic algorithm for multimodal function optimization, *Evolutionary Comput.* 10 (3) (2002) 207–234.
- [18] S.-C. Lin, W.F. Punch, E.D. Goodman, Coarse-grain genetic algorithms, categorization and new approaches, in: *Sixth IEEE Symposium on Parallel and Distributed Processing*, Dallas, TX, USA, October 1994, IEEE Computer Society Press, Silver Spring, MD, pp. 28–37.
- [19] Y. Liu, X. Yao, T. Higuchi, Evolutionary ensembles with negative correlation learning, *IEEE Trans. Evolutionary Comput.* 4 (4) (2000) 380–387.
- [20] R. McKay, H.A. Abbass, Anti-correlation: a diversity promoting mechanisms in ensemble learning, *Austral. J. Intell. Inform. Process. Systems* (3/4) (2001) 139–149.
- [21] N.F. McPhee, N.J. Hopper, Analysis of genetic diversity through population history, in: W. Banzhaf et al. (Eds.), *Proceedings of the Genetic and Evolutionary Computation Conference*, FL, USA, 1999. Morgan Kaufmann, Los Altos, CA, 1999, pp. 1112–1120.
- [22] C. Pettey, M. Leuze, J. Grefenstette, A parallel genetic algorithm, in: J.J. Grefenstette (Ed.), *Proceedings of the Second International Conference on Genetic Algorithms and Their Applications*, Hillsdale, NJ, USA, 1987, Lawrence Erlbaum Associates, Hillsdale, NJ, 1987.
- [23] M.A. Potter, K.A. De Jong, Cooperative coevolution: an architecture for evolving coadapted subcomponents, *Evolutionary Comput.* 8 (1) (2000) 1–29.
- [24] W.F. Punch, D. Zongker, E.D. Goodman, The royal tree problem, a benchmark for single and multi-population genetic programming, in: P.J. Angeline, K.E. Kinnear, Jr. (Eds.), *Advances in Genetic Programming* vol. 2, The MIT Press, Cambridge, MA, USA, 1996, pp. 299–316 (Chapter 15).
- [25] R. Tanese, Parallel genetic algorithms for a hypercube, in: J.J. Grefenstette (Ed.), *Proceedings of the Second International Conference on Genetic Algorithms and Their Applications*, Hillsdale, NJ, USA, 1987. Lawrence Erlbaum Associates, Hillsdale, NJ, 1987, pp. 177–183.



Steven Gustafson is a computer scientist at the General Electric Global Research Center in Niskayuna, New York. As a member of the Computational Intelligence Lab, he develops and applies advanced AI and machine learning algorithms for complex problem solving. He received his Ph.D. in computer science from the University of Nottingham, UK, where he was a research fellow in the Automated Scheduling, Optimisation and Planning Research Group. He received his BS and MS in computer science from Kansas State University. His Ph.D. dissertation was nominated for the British Computer Society and the Conference of Professors and Heads of Computing Distinguished Dissertation award, which recognizes the top Ph.D. thesis in the UK computer science community. In 2005 and 2006, he coauthored papers that won the Best Paper Award at the European Conference on Genetic Programming. He serves as a reviewer for several conferences, journals and recently was a co-editor of the Proceedings for the 2006 European Conference on Genetic Programming. In 2006, he was a recipient of the “IEEE Intelligent Systems 10 to Watch” in Artificial Intelligence award.



Edmund Burke is Head of the School of Computer Science and IT at the University of Nottingham. He also leads the Automated Scheduling, Optimisation and Planning Research (ASAP) Group and is Director of the Inter-disciplinary Optimisation Laboratory at Nottingham. He is a member of the Engineering and Physical Sciences (EPSRC) Peer Review College. Professor Burke is Editor-in-chief of the *Journal of Scheduling* (Springer), Area Editor (for Combinatorial Optimisation) of the *Journal of Heuristics* (Springer), Associate Editor of the *INFORMS Journal on Computing* and Associate Editor of the *IEEE Transactions on Evolutionary Computation*. He is also a guest co-editor of a feature issue of the *European Journal of Operational Research* (EJOR) on Timetabling and Rostering published in 2004 and of a further feature issue, to appear in 2006, on Evolutionary and Metaheuristic Scheduling. In addition, he is a guest co-editor of a forthcoming special issue of the *Annals of Operations Research* on Cutting, Packing and Layout.

He is chairman of the steering committee of the international series of conferences on the Practice and Theory of Automated Timetabling (PATAT). He has been co-chairman of the programme committee and co-editor of the conference proceedings and Selected Papers volumes (published by Springer) since 1995. This has covered 5 conferences (Edinburgh 1995, Toronto 1997, Konstanz 2000, Gent 2002 and Pittsburgh 2004). He will continue with these duties for the 2006 PATAT conference to be held in Brno (Czech Republic). He was Co-Chair of the Programme Committee of the international conference on Multi-disciplinary Scheduling: Theory and Applications (MISTA) held at Nottingham in August 2003. He was co-editor of the Proceedings of

the Genetic and Evolutionary Computation Conference (GECCO) in 2001, 2002 and 2004. He has also acted as Co-editor of the proceedings of the Parallel Problem Solving from Nature (PPSN) conferences in 2004 and 2006. Professor Burke is a member of the Scientific Committee of the Smith Institute for Industrial Mathematics and Systems Engineering. He is also a director of eventMAP Ltd. and Aptia Solutions Ltd., both of which are spin out companies from the ASAP group.

Prof. Burke has been a member of the Programme committees of over 70 international conferences in the last few years. He has edited/authored 12 books (with a further 2 in preparation) and has published over 130 refereed papers. He has also been awarded 41 externally funded grants worth over 8M from a variety of sources including EPSRC, ESRC, BBSRC, EU, East Midlands Development Agency, HEFCE, Teaching Company Directorate, Joint Information Systems Committee of the HEFCs and commercial organisations.